Meowcessor CPU Project

ALU Wrapper & Dispatch Design Document

Aadi

September 3, 2026

Contents

1 Introduction **2**

2 Architecture and Topology **2**
2.1 Sub-units 2
2.2 Topology Diagram 2

3 Opcode Integration (`defines.v`) **3**

4 Two-Phase Registered Dispatch (Timing Fix) **4**
4.1 The Problem 4
4.2 The Solution 4

5 Generated Gate-Level Hierarchy **5**

6 Conclusion **5**

1 Introduction

This document details the architecture and implementation of the Arithmetic Logic Unit (ALU) wrapper for the **Meowcessor** RISC-V 32-bit CPU. The ALU wrapper unifies the project's individual arithmetic units—specifically the Kogge-Stone parallel-prefix adder, the Booth radix-4 multiplier, and the SRT radix-4 divider—behind a single, cohesive interface.

The wrapper handles internal opcode decoding, dispatches operations to the correct sub-unit, manages multi-cycle operation handshaking, and routes the final results back to the execution pipeline.

2 Architecture and Topology

The ALU acts as a central multiplexer and orchestrator over several distinct execution units. Upon receiving a `start` pulse alongside a 4-bit internal `alu_op`, the dispatch unit categorizes the operation as either **single-cycle** or **multi-cycle**.

2.1 Sub-units

- **Kogge-Stone Adder (`Add`):** Purely combinational. The ALU reuses this block for `ADD`, `SUB` (computed as $a+ \sim b + 1$), and signed/unsigned comparisons (`SLT`, `SLTU`), where the subtraction result is inspected for sign bit and carry-out flags.
- **Bitwise Logic & Shifts:** Purely combinational blocks implementing native Verilog bitwise logic (`AND`, `OR`, `XOR`) and shifts (`SLL`, `SRL`, `SRA`).
- **Booth Multiplier (`booth_radix4_multiplier`):** A sequential unit with a latency of 17 cycles, responsible for `MUL` and `MULH`.
- **SRT Divider (`radix4_srt_divider`):** A sequential unit with a latency of approximately 18 cycles, responsible for `DIV`, `DIVU`, `REM`, and `REMU`.

2.2 Topology Diagram

The following is an abstract representation of the ALU routing topology:

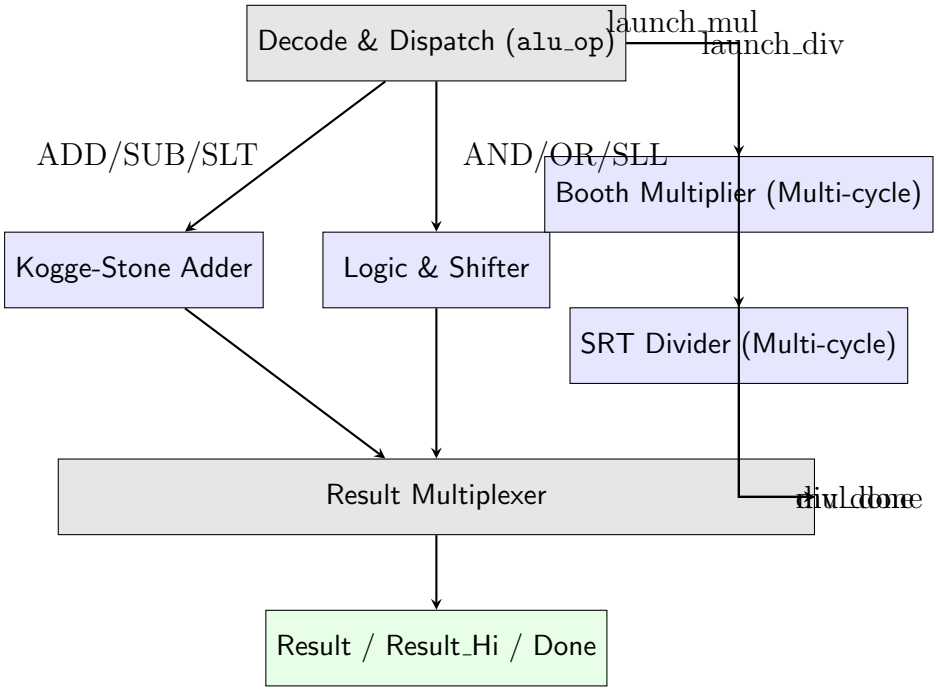


Figure 1: ALU Internal Routing Topology

3 Opcode Integration (`defines.v`)

To decouple the ALU from the complex RV32IM Instruction Decode stage, we map the standard RISC-V fields (`opcode`, `funct3`, `funct7`) into a compact 4-bit internal representation (`ALU_OP_*`). This acts as the exclusive command language for the ALU wrapper.

Internal Code	Decimal	Description	Execution Unit	Latency
ALU_OP_ADD	0	Addition	Kogge-Stone Adder	1
ALU_OP_SUB	1	Subtraction	Kogge-Stone Adder	1
ALU_OP_AND	2	Bitwise AND	Logic Block	1
ALU_OP_OR	3	Bitwise OR	Logic Block	1
ALU_OP_XOR	4	Bitwise XOR	Logic Block	1
ALU_OP_SLL	5	Logical Left Shift	Shifter Block	1
ALU_OP_SRL	6	Logical Right Shift	Shifter Block	1
ALU_OP_SRA	7	Arithmetic Right Shift	Shifter Block	1
ALU_OP_SLT	8	Set Less Than (Signed)	Kogge-Stone Adder	1
ALU_OP_SLTU	9	Set Less Than (Unsigned)	Kogge-Stone Adder	1
ALU_OP_MUL	10	Multiplication (Low)	Booth Multiplier	17
ALU_OP_MULH	11	Multiplication (High)	Booth Multiplier	17
ALU_OP_DIV	12	Division (Signed)	SRT Divider	∼18
ALU_OP_DIVU	13	Division (Unsigned)	SRT Divider	∼18
ALU_OP_REM	14	Remainder (Signed)	SRT Divider	∼18
ALU_OP_REMU	15	Remainder (Unsigned)	SRT Divider	∼18

Table 1: Meowcessor ALU Internal Opcodes

4 Two-Phase Registered Dispatch (Timing Fix)

A significant challenge encountered during integration was a delta-cycle race condition between the ALU wrapper's state machine and the internal state machines of the Multiplier/Divider.

4.1 The Problem

When the testbench (or ID stage) asserted `start = 1` for a single clock cycle, a naive combinational dispatch (`assign mul_start = start && is_multicycle`) caused a race. On the rising clock edge, the ALU wrapper registered its `multicycle_active` flag. Simultaneously, the sub-unit sampled `mul_start`. The glitch on the combinational wire resolved before the sub-unit could reliably capture the start pulse, causing the multiplier to remain idle.

4.2 The Solution

The ALU utilizes a registered two-phase approach to guarantee a clean handshake.

1. **Phase 1 (Cycle N):** The ALU detects `start == 1`. It asserts its internal `multicycle_active` state and registers a launch request (e.g., `launch_mul <= 1`).

2. **Phase 2 (Cycle N+1):** The `launch_mul` signal generates a clean, registered one-cycle pulse (`mul_start <= 1`). The sub-unit safely samples this on Cycle N+2.

This pattern adds exactly one cycle of latency to multi-cycle operations but guarantees robust synchronization across clock edges.

5 Generated Gate-Level Hierarchy

The ALU wrapper was synthesized using Yosys to verify elaboration and module connectivity. Below is the generated hierarchy schematic of the `alu` top module and its instantiations of the Kogge-Stone Adder, Booth Multiplier, and SRT Divider.

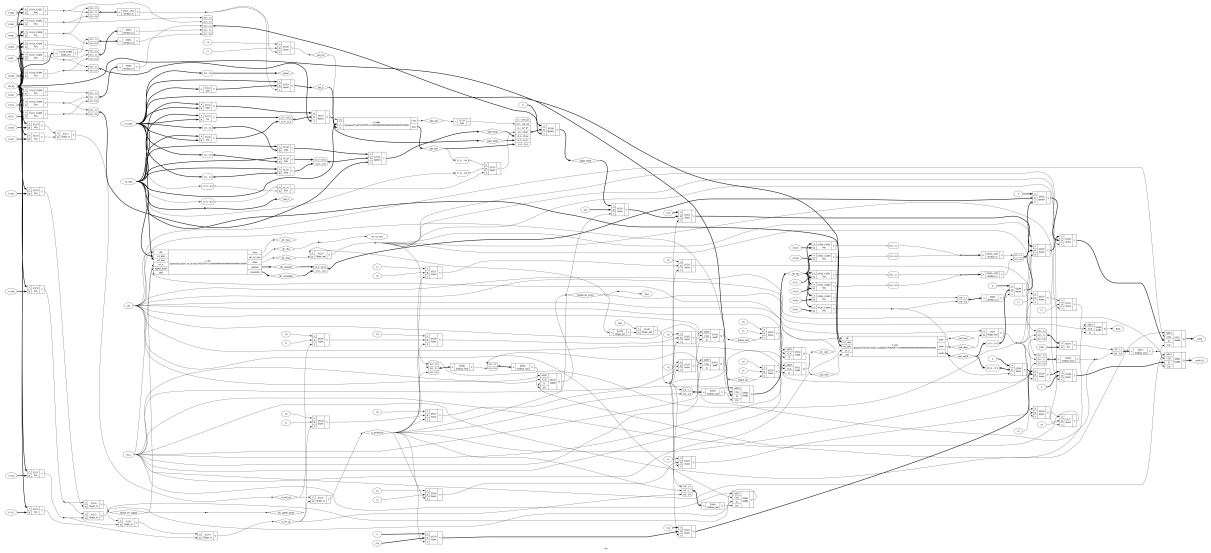


Figure 2: Yosys Generated Hierarchy Schematic

6 Conclusion

The completed ALU wrapper passes all 1,571 directed and randomized test vectors in the testbench (`alu_tb.v`). The module is now ready to be integrated with the CPU's Register File and Instruction Decode (ID) pipeline stages.